NLP LAB PROJECT · BIT MESRA

College Knowledge Assistant

A Retrieval-Augmented Generation (RAG) application for question answering over PDF documents

Python · LangChain · pypdf · Ollama (Llama 3.2 3B + nomic-embed-text) · FAISS · Streamlit

Team: [Member 1], [Member 2], [Member 3], [Member 4]

Guide: [Faculty Name]

Birla Institute of Technology, Mesra

Introduction & Problem Statement

Our project is a chatbot that answers questions about an uploaded PDF using Retrieval-Augmented Generation (RAG).

The problem

- College information (lecture notes, syllabi, notices) is spread across long PDFs
- Manual searching is slow; Ctrl+F matches exact words, not meaning
- General-purpose LLMs were never trained on our college documents
- So they may give confident but wrong answers (hallucination)

Our solution

- Upload a PDF and ask questions in plain English
- Relevant chunks are retrieved and given to a local LLM
- The answer uses only the PDF, and source pages are shown
- If the PDF lacks the answer, it replies “not available”
- Runs 100% locally with Ollama: no API key, data stays private

EXAMPLE FROM OUR TEST RUN (NLP LECTURE PDF)

Q: Who proposed Word2Vec and when?

A: “According to the document, Word2Vec was proposed by Mikolov et al. in 2013.” Sources: pages 10–13

What is Retrieval-Augmented Generation (RAG)?

RAG = search the document first, then let the LLM answer from what was found (like an open-book exam).

1 Retrieval

Find the document chunks most similar to the question (FAISS vector search).

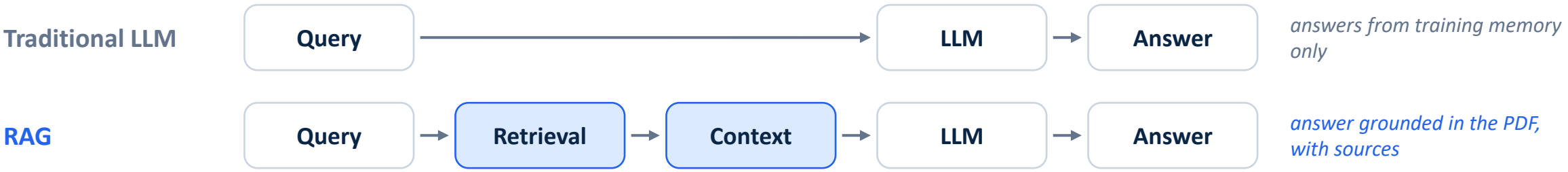
2 Augmentation

Add the retrieved chunks to the prompt as CONTEXT, next to the question.

3 Generation

The LLM generates the answer using only that context.

Traditional LLM vs RAG



Why RAG and not fine-tuning? No retraining; a new PDF works immediately; every answer can be traced to its source pages.

Technology Stack (as implemented)

Component	Technology used	Role in our project
Language	Python 3.14	rag_pipeline.py: RAG logic · app.py: user interface
PDF reading	pypdf (PdfReader)	Extracts text page by page; empty pages are skipped
Chunking	LangChain RecursiveCharacterTextSplitter	500-character chunks with 100-character overlap
Embeddings	nomic-embed-text via Ollama	Turns chunks and questions into 768-dimensional vectors
Vector search	FAISS (faiss-cpu, IndexFlatIP)	Exact cosine-similarity search; returns the Top-4 chunks
LLM	Llama 3.2 3B via Ollama (ChatOllama)	Writes the answer; temperature 0, num_ctx 4096
Orchestration	LangChain core (ChatPromptTemplate, LCEL)	Chain: prompt | llm | StrOutputParser
User interface	Streamlit	PDF upload, chat, source pages and retrieved text
Configuration	python-dotenv (.env file)	Model names, CHUNK_SIZE, CHUNK_OVERLAP, TOP_K

Note: both models run inside Ollama on the laptop (localhost:11434), so there is no API key and no internet use. We use the small LangChain packages (langchain-core, langchain-text-splitters, langchain-ollama), not langchain-community.

System Architecture & Working Pipeline

A. Document ingestion (once per uploaded PDF)

`process_pdf()`



same FAISS index, kept in st.session_state (PDF is not re-processed per question)

B. Question answering (every question)

`answer_question()`



For sample.pdf: 13 pages → 12 pages with text → 17 chunks. Each question sends 4 chunks (up to about 2,000 characters) to the LLM.

Core Technical Concepts

1. Embeddings

nomic-embed-text · makes vectors

- Converts text into a 768-number vector
- Similar meaning → vectors point in a similar direction
- Chunk input: “search_document: ” + text
- Question input: “search_query: ” + text
- Prefixes match how the model was trained

2. FAISS

IndexFlatIP · searches vectors

- Stores only vectors; FAISS id i ↔ chunks[i]
- All vectors L2-normalised to length 1
- So inner product = cosine similarity
- “Flat” = exact search against every vector
- Returns Top-K = 4 chunks with scores

3. LLM

llama3.2:3b · writes the answer

- Gets the question + 4 chunks tagged [Page N]
- Rule: use only facts in the CONTEXT
- No answer in context → fixed “not available” reply
- Ignores instructions that try to change its rules
- temperature 0 → factual, repeatable answers

Similarity: $\cos\theta = q \cdot d / (\|q\| \|d\|) = q \cdot d$ (q = question vector, d = chunk vector, both normalised to length 1)

Key point: the embedding model only makes vectors, FAISS only searches them, and the LLM only writes the answer.

Important Code (rag_pipeline.py)

1. Load and split the PDF

pypdf reads each page; chunks keep their page number

```
reader = PdfReader(pdf_file)
page_texts = [page.extract_text() or "" for page in reader.pages]
splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    separators=["\n\n", "\n", " ", ""],
)
chunks = splitter.split_documents(documents)
```

2. Embeddings + FAISS index and search

Prefixed chunks → vectors → IndexFlatIP; the query is searched the same way

```
texts = [DOCUMENT_PREFIX + chunk.page_content for chunk in chunks]
vectors = embedding_model.embed_documents(texts)
faiss.normalize_L2(vectors)               # length 1 -> IP = cosine
self.index = faiss.IndexFlatIP(dimension) # exact "flat" index
self.index.add(vectors)
query_vector = embed_question(question, self.embedding_model)
faiss.normalize_L2(query_vector)
scores, ids = self.index.search(query_vector, k)
```

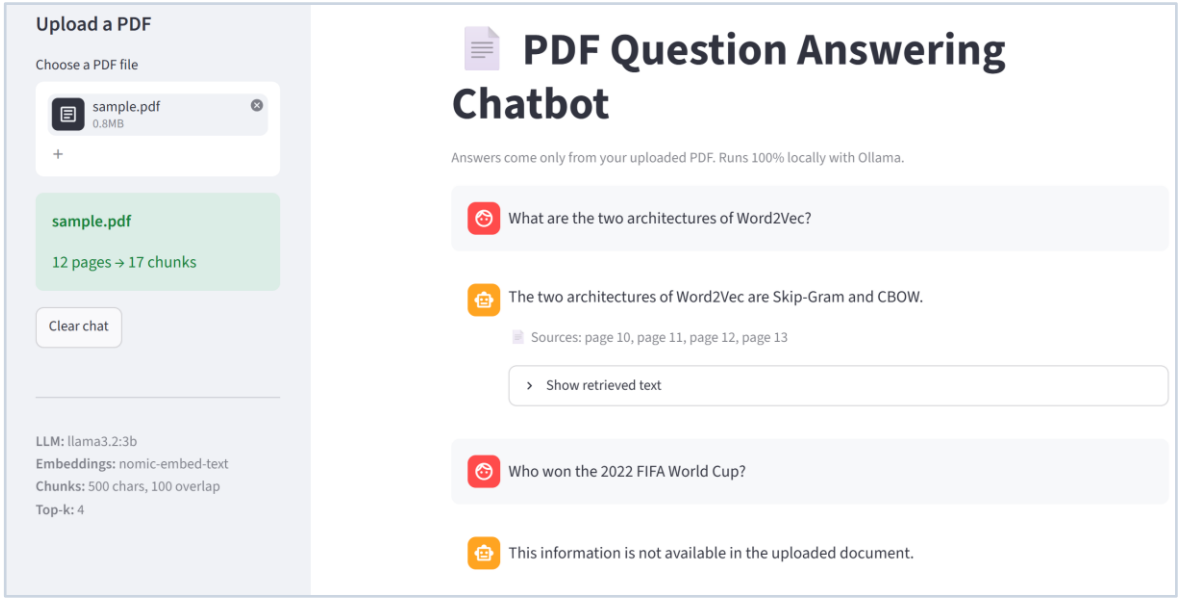
3. Prompt + LLM (LangChain LCEL chain)

Top-4 chunks fill {context}; prompt | llm | parser gives the answer

```
RAG_PROMPT = ChatPromptTemplate.from_messages([
    ("system",
     "You are a question-answering assistant ... "
     "1. Use only facts stated in the CONTEXT. ... "
     "2. If the CONTEXT does not contain the answer, ... "
     "reply with exactly this sentence ...: \"{not_found}\""
     ...),
    ("human", "CONTEXT:\n{context}\n\nQUESTION: {question}"),
])
# get_llm()
return ChatOllama(model=LLM_MODEL, base_url=OLLAMA_BASE_URL,
                  temperature=0, num_ctx=4096)
# answer_question()
chain = RAG_PROMPT | llm | StrOutputParser()
answer = chain.invoke({
    "context": context, "question": question,
    "not_found": NOT_FOUND_MESSAGE,
}).strip()
```

Guardrail: if the reply is the “not available” sentence, the app shows no sources.

Demo, Results & Limitations



Our Streamlit app: PDF uploaded and indexed (left), answer with source pages, and an out-of-scope question refused.

Limitations

- Scanned or image-only PDFs need OCR (not supported yet)
- Fixed 500-character chunks can split an idea in two
- A small 3B model can still misread or hallucinate
- Local inference adds latency; one PDF per session, index kept in memory

Self-test results on sample.pdf

13-page NLP lecture PDF → 12 pages with text → 17 chunks

Question asked	App's answer	Sources
What are the two architectures of Word2Vec?	Skip-Gram and CBOW	p. 10–13
What is the difference between TF-IDF and one-hot encoding?	IDF down-weights common words; one-hot has no semantics	p. 4, 6, 9
Who proposed Word2Vec and when?	Mikolov et al., 2013	p. 10–13
Who won the 2022 FIFA World Cup?	“Not available” (refused)	none
What is the capital of France?	“Not available” (refused)	none

Top-4 similarity scores (Word2Vec question): **0.72, 0.71, 0.70, 0.69** (pages 12, 13, 11, 10)

3 of 3 in-document questions answered consistently with the PDF; 2 of 2 out-of-scope questions refused. This is a small manual check, not a formal benchmark.

Future Scope & Conclusion

Future scope

- **More formats:** DOCX, PPTX and scanned PDFs (OCR)
- **Multiple PDFs:** one shared index, saved to disk
- **Better retrieval:** hybrid keyword + vector search, reranking
- **Conversation memory:** follow-up questions using chat history
- **Finer citations:** sentence-level (page-level already shown)
- **Evaluation & deployment:** labelled test set; campus server

Conclusion

- Built a complete RAG pipeline: PDF → chunks → embeddings → FAISS → LLM
- Answers come only from the uploaded document, with source pages
- Out-of-scope questions are refused instead of guessed
- Runs fully locally with open models; no API key needed

Thank You | Questions & Answers